Majority Element in an Array

Using Pointers and Dynamic Memory Allocation in C

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

CONTENTS

1. Problem Statement
2. Objective
3. Concepts Used
4. Algorithm
5. Flowchart
6. Source Code
7. Sample Input and Output
8. Working of the Program (Dry Run)
9. Time and Space Complexity
10. Conclusion

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array, nums[]. The array must be passed to a function named majority_element() using its address, and pointers must be used to pass the array between the calling function and majority_element(). The function must:

1. Count the occurrences of each distinct element in nums[].
2. Find the element that appears strictly more than ⌊ size / 2⌋ times.
3. Display the majority element if it exists.
4. Display -1 if no majority element exists.

Note: The majority element is not necessarily the element with the highest frequency. It must occur strictly more than ⌊ size / 2⌋ times.

2. Objective

The objective of this project is to design and implement a C program that correctly identifies the majority element of an integer array whose size is decided by the user at run time. The project also demonstrates the use of pointers, pointer arithmetic, dynamic memory allocation and modular programming through functions.

3. Concepts Used

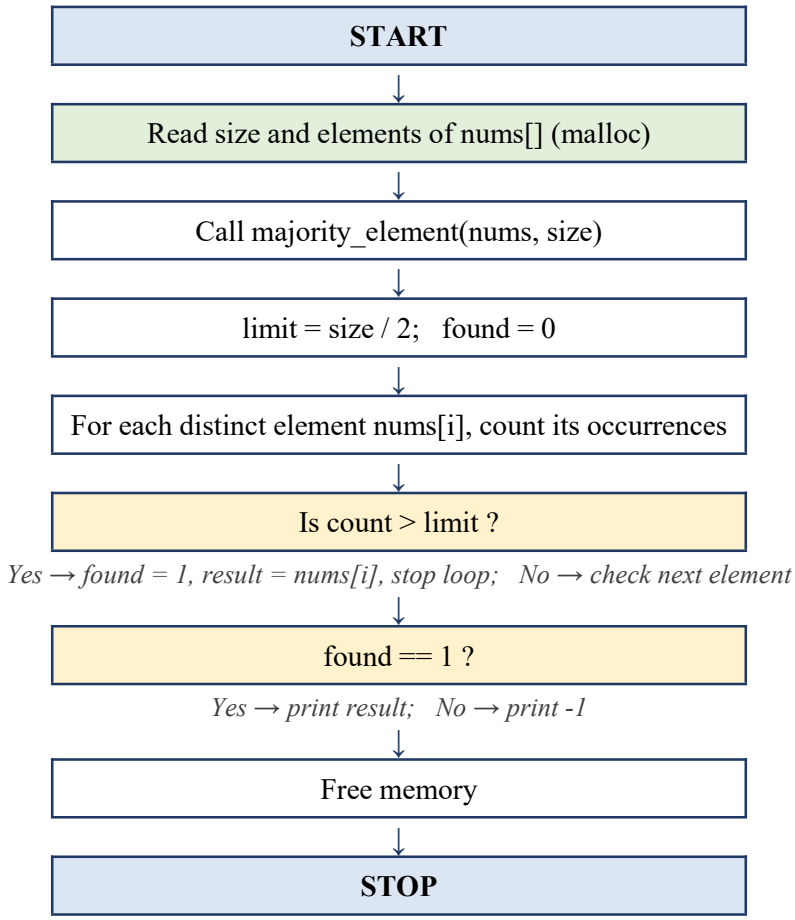
- **Pointers:** The array is accessed and passed using its base address (int *nums).
- **Pointer arithmetic:** Elements are read and compared using *(nums + i) instead of nums[i].
- **Dynamic memory allocation:** malloc() creates the array of the size entered by the user, and free() releases the memory.
- **Functions:** The logic is separated into the function majority_element().
- **Loops and conditions:** Nested for loops count the occurrences of each distinct element.

4. Algorithm

1. Start.
2. Read the size of the array and store it in size. If size is not a positive integer, display an error message and stop.
3. Allocate memory for size integers using malloc() and read the elements into nums using pointers.
4. Call majority_element(nums, size), passing the base address of the array.

5. Inside the function, calculate limit = size / 2 (integer division gives ⌊ size / 2 ⌋).
6. For each element nums[i], check whether it has already appeared earlier in the array. If yes, skip it.
7. Otherwise, count how many times nums[i] occurs in the whole array.
8. If this count is strictly greater than limit, then nums[i] is the majority element. Store it and stop searching.
9. After the loop, if a majority element was found, display it. Otherwise display -1.
10. Free the allocated memory in main().
11. Stop.

5. Flowchart



6. Source Code

File name: majority_element.c

```
#include <stdio.h>
#include <stdlib.h>

/* Function prototype: the array is received through a pointer */
void majority_element(int *nums, int size);

int main()
{
    int size, i;
    int *nums;

    printf("Enter the size of the array: ");
    if (scanf("%d", &size) != 1 || size <= 0)
    {
        printf("Invalid size. Size must be a positive integer.\n");
        return 1;
    }

    /* Allocate memory for a variable-length array */
    nums = (int *)malloc(size * sizeof(int));
    if (nums == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", nums + i);          /* store using pointer */
    }

    /* Pass the base address of the array to the function */
    majority_element(nums, size);

    free(nums);                          /* release memory */
    return 0;
}

/* Finds and displays the majority element (count > size/2), else -1 */
void majority_element(int *nums, int size)
{
    int i, j, count;
    int limit = size / 2;               /* floor(size / 2) */
    int found = 0;
    int result = -1;

    for (i = 0; i < size; i++)
    {
        /* Skip the element if it was already counted earlier */
        int already_counted = 0;
        for (j = 0; j < i; j++)
        {
            if (*(nums + j) == *(nums + i))
            {
                already_counted = 1;
                break;
```

```
            }
        }
        if (already_counted)
            continue;

        /* Count occurrences of this distinct element */
        count = 0;
        for (j = 0; j < size; j++)
        {
            if (*(nums + j) == *(nums + i))
                count++;
        }

        /* Check the majority condition: strictly greater than floor(size/2) */
        if (count > limit)
        {
            found = 1;
            result = *(nums + i);
            break;
        }
    }

    if (found)
        printf("Majority element: %d\n", result);
    else
        printf("-1\n");
}
```

7. Sample Input and Output

Test Case 1

```
Enter the size of the array: 5
Enter 5 elements: 3 3 2 3 3
Majority element: 3
```

The element 3 occurs four times. Here, ⌊ 5 / 2⌋ = 2. Since 4 > 2, the element 3 is the majority element.

Test Case 2

```
Enter the size of the array: 7
Enter 7 elements: 2 2 1 1 1 2 2
Majority element: 2
```

The element 2 occurs four times. Here, ⌊ 7 / 2⌋ = 3. Since 4 > 3, the element 2 is the majority element.

Test Case 3

```
Enter the size of the array: 6
Enter 6 elements: 1 2 3 2 3 3
-1
```

The element 3 occurs three times. Here, ⌊ 6 / 2⌋ = 3. Since 3 is not strictly greater than 3, no majority element exists and -1 is displayed.

Test Case 4 (Single element)

```
Enter the size of the array: 1
Enter 1 elements: 7
Majority element: 7
```

With size = 1, ⌊ 1 / 2⌋ = 0. The element 7 occurs once, and 1 > 0, so 7 is the majority element.

Test Case 5 (Two different elements)

```
Enter the size of the array: 2
Enter 2 elements: 4 5
-1
```

With size = 2, ⌊ 2 / 2⌋ = 1. Each element occurs only once, which is not strictly greater than 1, so -1 is displayed.

8. Working of the Program (Dry Run)

Consider the input of Test Case 1: size = 5, nums = {3, 3, 2, 3, 3}. Here limit = ⌊ 5 / 2⌋ = 2.

i	nums[i]	Already counted?	Count	count > 2 ?
0	3	No	4	Yes
—	—	Loop stops as the majority element is found	—	—

Since the condition is satisfied for the element 3 at i = 0, found is set to 1 and result = 3. The program therefore prints: Majority element: 3.

Now consider Test Case 3: size = 6, nums = {1, 2, 3, 2, 3, 3}. Here limit = ⌊ 6 / 2⌋ = 3.

i	nums[i]	Already counted?	Count	count > 3 ?
0	1	No	1	No
1	2	No	2	No
2	3	No	3	No
3	2	Yes (skipped)	—	—
4	3	Yes (skipped)	—	—
5	3	Yes (skipped)	—	—

No element has a count greater than 3, so found remains 0 and the program prints -1.

9. Time and Space Complexity

- **Time complexity:** $O(n^2)$, because for each distinct element the whole array is scanned once to count its occurrences (and earlier elements are checked to avoid recounting).

- **Space complexity:** O(1) extra space apart from the input array, since only a few integer variables are used.

10. Conclusion

The program successfully reads an array of any size entered by the user, passes it to majority_element() using pointers, counts the occurrences of every distinct element and checks the condition count > ⌊ size / 2⌋ . It displays the majority element when it exists and -1 otherwise. All the given examples, along with additional edge cases, were compiled with gcc and tested, and produced the expected output. Through this project, the concepts of pointers, pointer arithmetic, dynamic memory allocation and functions in C were practised.